07

DEEP DIVE 07 · NOTIFICATION SERVICE · MESSAGE QUEUE · EVENT BUS

Design a Pub-Sub System

> Uber · Goldman Sachs · Walmart · Eightfold AI — ఇది "Notification Service", "Message Queue", "Event Bus" అని మూడు పేర్లతో వస్తుంది. మూడూ ఒకే problem.

మొదటి version పది పంక్తుల్లో రాయొచ్చు, మరియు అది **మూడుసార్లు** విరుగుతుంది – ఒక subscriber విఫలమైతే మిగతావాళ్ళు కోల్పోతారు; ఒక నెమ్మది subscriber publisher ని ఆపేస్తాడు; మరియు ఒక crash message ని **శాశ్వతంగా** మాయం చేస్తుంది. మూడూ నడిపి చూపిస్తాను.

తెలుగు + ENGLISH

YASWANTH · DEEP DIVE 07

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి **ప్రతి output, ప్రతి timing నిజంగా `node` lo run చేసినదే**.

> **OBSERVER PATTERN నా, MESSAGE BROKER నా – ఈ తేడా ముందే తెలియాలి**
>
> GoF **Observer** మరియు ఒక **message broker** – రెండూ "ఒకరు ప్రకటిస్తారు, చాలామంది వింటారు" అని చేస్తాయి. కానీ అవి ఒకటి కాదు:
>
> **Observer** – ఒకే process లోపల, synchronous, listeners publisher యొక్క thread lo నడుస్తారు, మరియు వాళ్ళ జీవితకాలం publisher తో ముడిపడి ఉంటుంది. `addEventListener` ఇదే.
>
> **Broker** – publisher మరియు subscriber ఒకరి గురించి ఒకరు ఏమీ తెలియనవసరం లేదు, వేర్వేరు సమయాల్లో నడవొచ్చు, మరియు messages **నిలిచి ఉంటాయి**. Kafka, SQS, RabbitMQ ఇవే.
>
> ఈ doc lo మనం **Observer తో మొదలుపెట్టి, broker కి పెరుగుతాం** – ఎందుకంటే ప్రతి విరుపూ ఆ దూరాన్ని ఒక అడుగు తగ్గిస్తుంది. Interview lo ఈ తేడాని మొదట్లోనే చెప్పడం ఒక బలమైన ప్రారంభం.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. ఇది ఎందుకు కావాలి – నేరుగా పిలిస్తే ఏమవుతుంది
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: ఒకరు పడితే అందరూ

3. Step – పది పంక్తుల event bus
4. **మొదటి విరుపు** – ఒక subscriber విసిరితే మిగతావాళ్ళు కోల్పోతారు
5. Step – ప్రతి subscriber ని వేరుచేయడం

Part 3 – రెండో విరుపు: నెమ్మది subscriber

6. **రెండో విరుపు** – publisher 415 ms ఎదురుచూసింది
7. Step – ప్రతి subscriber కి సొంత queue

Part 4 – మూడో విరుపు: మాయమైన message

8. **మూడో విరుపు** – ఒక crash, ఒక payment శాశ్వతంగా పోయింది
9. Step – Ack, retry, మరియు DLQ
10. At-most-once vs at-least-once – ఒక ఎంపిక, రెండు ఖర్చులు

Part 5 – పూర్తి system

11. Step – Topics మరియు filters

12. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

Part 6 – Interview lo

13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
14. నోటితో చెప్పాల్సిన English script
15. Follow-ups – ordering, fan-out, Kafka
16. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

---

1. ఇది ఎందుకు కావాలి – నేరుగా పిలిస్తే ఏమవుతుంది

ఒక e-commerce app. ఒక order place అయింది. ఇప్పుడు ఏమి జరగాలి?

- Confirmation email పంపాలి
- SMS పంపాలి
- Inventory తగ్గించాలి
- Analytics lo నమోదు చేయాలి
- Fraud check నడపాలి
- Warehouse కి చెప్పాలి

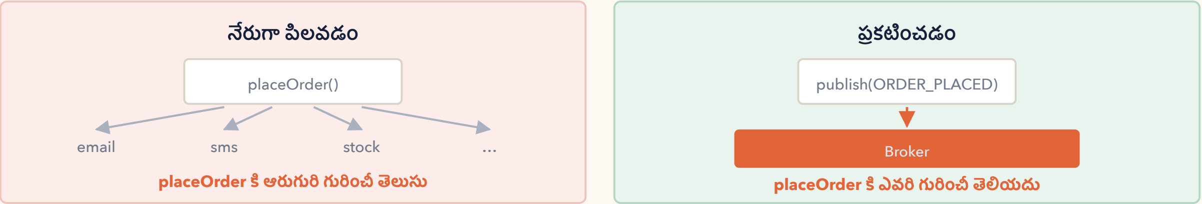
సులభమైన జవాబు – `placeOrder()` లోపల అన్నింటినీ పిలవడం:

```
function placeOrder(order) {
  saveOrder(order);
  emailService.sendConfirmation(order);
  smsService.send(order);
  inventory.reduce(order);
  analytics.track(order);
  fraudCheck.run(order);
  warehouse.notify(order);
}
```

ఇది పనిచేస్తుంది. కానీ మూడు సమస్యలు ఉన్నాయి, మరియు అవి కాలంతో పెరుగుతాయి:

నేరుగా పిలవడం VS ప్రకటించడం

ఒక order place అయినప్పుడు ఆరు పనులు జరగాలి



నేరుగా పిలవడం వల్ల వచ్చే మూడు సమస్యలు

1 · **కలయిక** – కొత్త పని చేర్చాలంటే placeOrder ని మార్చాలి. ఏడో పనికీ, ఎనిమిదోదానికీ మళ్ళీ.

2 · **వైఫల్యం** – SMTP పడిపోతే order place కాలేదని user కి కనిపిస్తుంది. కానీ order అయిపోయింది!

3 · **వేగం** – ఆరు పనులూ వరుసగా జరిగేదాకా user ఎదురుచూస్తాడు (§6 lo కొలుస్తాం).

ఈ మూడూ Pub-Sub పరిష్కరించేవే – కానీ **ఒక్కసారిగా కాదు**. ఈ doc అదే ప్రయాణం.

---

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
ఒకే process lo నా, పలు services మధ్యనా?	ఒకే process అంటే Observer చాలు. పలు services అంటే నిజమైన broker
Message పోతే పర్వాలేదా?	"పర్వాలేదు" (analytics) vs "అస్సలు కాదు" (payment) – ఇదే §10
ఒకే message రెండుసార్లు వస్తే పర్వాలేదా?	ఇది పై ప్రశ్న యొక్క వెనుక వైపు. **రెండూ ఒకేసారి సాధ్యం కాదు**
క్రమం ముఖ్యమా?	"order created" తర్వాతే "order shipped" రావాలా?
Subscriber నెమ్మదిగా ఉంటే?	Publisher ఆగాలా, లేక queue పెరగాలా? (§6)
ప్రతి subscriber కీ అన్ని messages కావాలా?	లేకపోతే **filtering** (§11)
Subscriber పడిపోయి తిరిగి వస్తే – పాత messages కావాలా?	"అవును" అంటే messages నిలిచి ఉండాలి (durable)

రెండు మూడు ప్రశ్నలు ఒకే నాణెం

"Message పోతే పర్వాలేదా?" మరియు *"రెండుసార్లు వస్తే పర్వాలేదా?"* – ఈ రెండిటికీ **"కాదు"** అని చెప్పలేరు. Distributed system lo **exactly-once delivery అసాధ్యం**.

మీరు ఎంచుకోవాల్సింది: **at-most-once** (పోవచ్చు, కానీ రెండుసార్లు రాదు) లేదా **at-least-once** (రెండుసార్లు రావచ్చు, కానీ పోదు). §10 lo దీని గురించి వివరంగా.

Interview lo ఈ తేడాని మీరే లేవనెత్తితే – అది ఒక పెద్ద సంకేతం, ఎందుకంటే ఇది distributed systems యొక్క మౌలిక వాస్తవం.

Part 2 – మొదటి విరుపు: ఒకరు పడితే అందరూ

3. Step – పది పంక్తుల event bus

అతి సులభమైన రూపం. ఒక జాబితా, ఒక loop:

```
class EventBus {
  constructor() { this.subscribers = []; }
  subscribe(fn) { this.subscribers.push(fn); }
  publish(event) { for (const fn of this.subscribers) fn(event); }
}
```

నడిపి చూద్దాం:

```
const bus = new EventBus();
bus.subscribe((e) => console.log(' 📧 Email పంపాం:', e.orderId));
bus.subscribe((e) => console.log(' 📊 Analytics నమోదు:', e.orderId));
bus.subscribe((e) => console.log(' 📦 Inventory తగ్గించాం:', e.orderId));
bus.publish({ type: 'ORDER_PLACED', orderId: 'ORD-1' });
```

```
ORDER_PLACED publish చేస్తున్నాం:
  📧 Email పంపాం: ORD-1
  📊 Analytics నమోదు: ORD-1
  📦 Inventory తగ్గించాం: ORD-1
```

పనిచేస్తోంది, మరియు §1 lo ఉన్న **కలయిక** సమస్య పరిష్కారమైంది – `placeOrder` కి ఇప్పుడు email, analytics, inventory గురించి ఏమీ తెలియదు. కొత్త subscriber చేర్చాలంటే ఒక `subscribe` పిలుపు, అంతే.

ఇది పది పంక్తులు, మరియు ఇది నిజమైన విలువ ఇచ్చింది. కాబట్టి ఇక్కడ ఆగిపోవచ్చా?

4. మొదటి విరుపు – ఒక subscriber విసిరితే మిగతావాళ్ళు కోల్పోతారు

ఇప్పుడు ఒక నిజమైన పరిస్థితి. SMTP server పడిపోయింది, కాబట్టి SMS subscriber ఒక error విసురుతుంది:

```
bus.subscribe((e) => console.log(' 📧 Email పంపాం:', e.orderId));
bus.subscribe((e) => { throw new Error('SMTP server down'); });     // ← ఇది విఫలమైంది
```

```
bus.subscribe((e) => console.log('📊 Analytics నమోదు:', e.orderId));
bus.subscribe((e) => console.log('📦 Inventory తగ్గించాం:', e.orderId));
```

```
ORDER_PLACED publish చేస్తున్నాం:
  📧 Email పంపాం: ORD-2
  💥 publish() విసిరింది: SMTP server down
=> Analytics, Inventory రెండూ ఎప్పటికీ తెలుసుకోలేదు.
=> మరియు order place చేసిన code కి ఒక SMTP error వచ్చింది.
```

మొదటి విరుపు – రెండు వేర్వేరు నష్టాలు

1 · Analytics మరియు Inventory ఎప్పటికీ తెలుసుకోలేదు. ఆ loop రెండో subscriber దగ్గరే ఆగిపోయింది. Inventory తగ్గలేదు – అంటే ఆ వస్తువుని ఇంకొకరికి కూడా అమ్మేస్తాం.

2 · Error publisher కి చేరింది. `placeOrder()` ఇప్పుడు ఒక *SMTP* error చూస్తోంది. User కి "order failed" అని కనిపిస్తుంది – కానీ order **నిజంగా save అయింది**. ఇప్పుడు అతను మళ్ళీ order చేస్తాడు, మరియు మీకు రెండు orders ఉంటాయి.

మౌలిక తప్పు: subscribers ఒకరి విధిని ఒకరు పంచుకుంటున్నారు, మరియు publisher వాళ్ళందరి విధిని పంచుకుంటున్నాడు. **Pub-Sub యొక్క మొత్తం ఉద్దేశమే వాళ్ళని వేరుచేయడం** – కానీ మన code అది చేయట్లేదు.

---

5. Step – ప్రతి subscriber ని వేరుచేయడం

పరిష్కారం స్పష్టం: ప్రతి subscriber పిలుపునీ ఒక `try/catch` lo చుట్టడం. ఒకరి వైఫల్యం ఇంకొకరిని తాకకూడదు.

```
publish(event) {
  const failures = [];
  for (const s of this.subscribers) {
    try { s.fn(event); }
    catch (err) { failures.push({ subscriber: s.name, error: err.message }); }
  }
  return { delivered: this.subscribers.length - failures.length, failures };
}
```

గమనించండి – errors ని **మింగట్లేదు**, వాటిని **సేకరించి తిరిగి ఇస్తున్నాం**. Publisher వాటిని log చేయొచ్చు, alert పంపొచ్చు. కానీ అతని స్వంత పని ఆగదు.

`subscribe` కి ఒక **పేరు** కూడా చేర్చాం – ఎందుకంటే "ఒక subscriber విఫలమైంది" అనేది పనికిరాని సమాచారం; "sms subscriber విఫలమైంది" అనేది ఉపయోగకరం.

```
ORDER_PLACED:
  📧 Email: ORD-3
  📊 Analytics: ORD-3
  📦 Inventory: ORD-3
```

```
  {
  delivered: 3,
  failures: [ { subscriber: 'sms', error: 'SMTP server down' } ]
}
```

మూడూ డెలివరీ అయ్యాయి, మరియు వైఫల్యం ఒక నివేదికగా మారింది, ఒక పేలుడు గా కాదు. మొదటి విరుపు సరిచేయబడింది.

Part 3 — రెండో విరుపు: నెమ్మది subscriber

6. రెండో విరుపు — publisher 415 ms ఎదురుచూసింది

Subscribers ఇప్పుడు ఒకరినొకరు విరగ్గొట్టట్లేదు. కానీ వాళ్ళు ఇంకా **publisher యొక్క సమయాన్ని** తింటున్నారు.

Fraud check ఒక ML model నడుపుతుంది — అది 400 ms తీసుకుంటుంది. మిగతా మూడూ చిన్నవి (5 ms). కొలుద్దాం:

```
bus.subscribe('email',       () => sleep(5));
bus.subscribe('analytics',   () => sleep(5));
bus.subscribe('fraud-check', () => sleep(400));    // ← ఈ ఒక్కటి నెమ్మది
bus.subscribe('inventory',   () => sleep(5));

const t = Date.now();
bus.publish({ orderId: 'ORD-4' });
console.log(`publish() తిరిగి రావడానికి పట్టిన సమయం: ${Date.now() - t} ms`);
```

```
publish() తిరిగి రావడానికి పట్టిన సమయం: 415 ms
=> user ఆ 400ms అంతా "Placing order…" చూస్తూ ఎదురుచూస్తున్నాడు.
=> నిజానికి order ఎప్పుడో place అయింది — fraud-check కోసం ఆగడం అనవసరం.
```

రెండో విరుపు — వేగం అత్యంత నెమ్మది SUBSCRIBER దే

`publish()` ఇప్పుడు **అందరి మొత్తం సమయం** తీసుకుంటోంది. అంటే — ఒక కొత్త subscriber చేర్చినప్పుడల్లా, మీ order API **నెమ్మదవుతుంది**.

ఇది ఒక దుష్ట ప్రోత్సాహకాన్ని సృష్టిస్తుంది: teams కొత్త subscriber చేర్చడానికి భయపడతారు, ఎందుకంటే అది checkout ని నెమ్మది చేస్తుంది. **Pub-Sub యొక్క ఉద్దేశమే** "స్వేచ్ఛగా subscriber చేర్చగలగడం" — ఆ ఉద్దేశం ఇక్కడ ఓడింది.

మౌలిక తప్పు: *ప్రకటించడం* మరియు *ప్రాసెస్ చేయడం* — ఈ రెండూ ఒకే thread lo, ఒకే క్షణంలో జరుగుతున్నాయి. అవి వేరు కావాలి.

7. Step – ప్రతి subscriber కి సొంత queue

పరిష్కారం: `publish()` **handler ని పిలవకూడదు**. అది కేవలం message ని ప్రతి subscriber యొక్క **queue lo పెట్టాలి**. ప్రాసెస్ చేయడం తర్వాత, విడిగా జరుగుతుంది.

```
class Broker {
  #queues = new Map();                        // subscriber → పెండింగ్ messages
  #handlers = new Map();

  subscribe(name, fn) { this.#handlers.set(name, fn); this.#queues.set(name, []); }

  publish(event) {
    for (const q of this.#queues.values()) q.push(event);   // ← కేవలం వరుసలో పెట్టడం
    return { queued: this.#queues.size };
  }

  // తర్వాత, విడిగా — ఒక "worker" ఈ queue ని ఖాళీ చేస్తుంది
  drain(name) {
    const q = this.#queues.get(name), fn = this.#handlers.get(name);
    let done = 0;
    while (q.length) { const e = q.shift(); try { fn(e); done++; } catch {} }
    return done;
  }
}
```

అదే పరీక్ష:

```
publish() సమయం: 0 ms   ← user ఇక ఎదురుచూడట్లేదు
పెండింగ్: email:1  fraud-check:1  inventory:1
ఇప్పుడు workers నడుస్తాయి (user కి సంబంధం లేకుండా):
  email: 1 message(s) ప్రాసెస్ అయ్యాయి
  fraud-check: 1 message(s) ప్రాసెస్ అయ్యాయి
  inventory: 1 message(s) ప్రాసెస్ అయ్యాయి
```

415 ms → 0 ms. Fraud check ఇప్పటికీ 400 ms తీసుకుంటుంది, కానీ అది **user యొక్క సమయంలో కాదు**.

ఇక్కడ ఒక ముఖ్యమైన మార్పు జరిగింది, మరియు అది code కంటే పెద్దది. ఇప్పటిదాకా మన system **synchronous** – publish అయిన క్షణం అంతా అయిపోయింది. ఇప్పుడు అది **asynchronous** – publish అంటే "ఇది జరుగుతుంది" అనే వాగ్దానం మాత్రమే.

అంటే మనం **Observer నుంచి broker వైపు** ఒక పెద్ద అడుగు వేశాం. మరియు దానితో పాటు ఒక కొత్త ప్రశ్న వచ్చింది: **ఆ వాగ్దానం ఎంత బలమైనది?** తర్వాతి Part అదే.

Part 4 – మూడో విరుపు: మాయమైన message

8. మూడో విరుపు – ఒక crash, ఒక payment శాశ్వతంగా పోయింది

`drain()` ని జాగ్రత్తగా చూడండి:

```
while (q.length) {
  const e = q.shift();          // ← ముందే queue నుంచి తీసేశాం
  try { fn(e); done++; } catch {}   // ← విఫలమైతే? అది ఇక ఎక్కడా లేదు.
}
```

`shift()` ఆ message ని queue నుంచి **శాశ్వతంగా తీసేస్తుంది**. ఆ తర్వాత handler విఫలమైతే – message **ఎక్కడా లేదు**. Queue lo లేదు, ప్రాసెస్ కాలేదు, ఎవరికీ తెలియదు.

Payment subscriber ORD-7 మీద విఫలమవుతుందని అనుకుందాం:

```
b.subscribe('payment', (e) => { if (e.orderId === 'ORD-7') throw new Error('DB
timeout'); });
b.publish({ orderId:'ORD-6' }); b.publish({ orderId:'ORD-7' }); b.publish({
orderId:'ORD-8' });
```

```
మూడు messages queue lo. drain చేస్తున్నాం…
  విజయవంతమైనవి: 2
  queue lo మిగిలినవి: 0
=> ORD-7 payment ప్రాసెస్ కాలేదు, మరియు queue lo కూడా లేదు.
=> ఆ order యొక్క payment *శాశ్వతంగా* పోయింది. ఎవరికీ తెలియదు.
```

మూడో విరుపు – నిశ్శబ్దంగా మాయమైన డబ్బు

ORD-7 యొక్క payment ఎప్పటికీ ప్రాసెస్ కాలేదు. Queue ఖాళీగా ఉంది, కాబట్టి system **అంతా బాగుంది** అనుకుంటుంది. ఏ alert లేదు, ఏ error లేదు.

Customer డబ్బు కట్టాడు, order confirm అయింది, కానీ payment record ఎక్కడా లేదు. ఇది నెలల తర్వాత reconciliation lo బయటపడుతుంది – అప్పటికి వందల orders అలా ఉంటాయి.

మౌలిక తప్పు: మనం message ని **ప్రాసెస్ చేయడానికి ముందే** తీసేశాం. సరైనది – ప్రాసెస్ *విజయవంతంగా* పూర్తయ్యాక మాత్రమే తీసేయడం.

9. Step – Ack, retry, మరియు DLQ

పరిష్కారం మూడు భాగాలు:

1. `poll()` **– తీసుకోవడం, కానీ వదలకపోవడం.** Message queue నుంచి బయటికి వస్తుంది, కానీ ఒక **in-flight** జాబితాలోకి వెళ్తుంది. అది ఇంకా broker చేతిలోనే ఉంది.

2. `ack()` **– పూర్తయింది.** Handler విజయవంతమైతే, message ని in-flight నుంచి తీసేయడం. ఇప్పుడు అది నిజంగా పోయింది.

3. `nack()` **– విఫలమైంది.** Message ని queue lo **తిరిగి పెట్టడం**, మరియు `attempts` ని పెంచడం. కొన్ని ప్రయత్నాల తర్వాత కూడా విఫలమైతే – దాన్ని **DLQ** (dead letter queue) lo పెట్టడం.

```
poll() {
  const m = this.#queue.shift();
  if (m) this.#inFlight.set(m.id, m);          // తీసుకున్నాం, కానీ ఇంకా వదల్లేదు
  return m ?? null;
}

ack(id) { this.#inFlight.delete(id); }          // పూర్తయింది → ఇప్పుడు వదలొచ్చు

nack(id) {
  const m = this.#inFlight.get(id);
  if (!m) return null;
  this.#inFlight.delete(id);
  m.attempts++;
  if (m.attempts >= this.maxAttempts) { this.dlq.push(m); return 'DLQ'; }
  this.#queue.unshift(m);                       // క్రమం కాపాడటానికి ముందే
  return 'RETRY';
}
```

ఒక MESSAGE యొక్క ప్రయాణం · నాలుగు గమ్యాలు

poll తర్వాత message ఇంకా broker చేతిలోనే ఉంటుంది – ఇదే మొత్తం ఉపాయం

DLQ ఎందుకు అవసరం

Retry లేకపోతే – message పోతుంది (§8). **అపరిమిత** retry అయితే – ఒక చెడ్డ message శాశ్వతంగా తిరుగుతూ queue ని నింపేస్తుంది, మరియు దాని వెనక అందరూ ఆగిపోతారు.

DLQ అంటే: **"ఇది మూడుసార్లు విఫలమైంది; ఇది ఒక తాత్కాలిక సమస్య కాదు."** అది పక్కన పెట్టి, మిగతా queue కదులుతుంది, మరియు ఒక మనిషి దాన్ని చూస్తాడు.

ORD-7 ని మళ్ళీ నడుపుదాం – ఈసారి ack/retry/DLQ తో:

```
payment (ORD-7 ఎప్పుడూ విఫలం):
  round 1: { ok: 1, retried: 1, dead: 0 } | pending 1
  round 2: { ok: 0, retried: 1, dead: 0 } | pending 1
  round 3: { ok: 0, retried: 0, dead: 1 } | pending 0
  round 4: { ok: 0, retried: 0, dead: 0 } | pending 0
```

```
మొత్తం ప్రయత్నాలు: 4
DLQ: 1 message(s)
```

ORD-6 విజయవంతమైంది (round 1 lo `ok: 1`). **ORD-7 మూడుసార్లు ప్రయత్నించి DLQ కి వెళ్ళింది.** ఇప్పుడు అది **మాయం కాలేదు** – అది ఒక నిర్దిష్ట చోట, ఒక మనిషి చూడటానికి సిద్ధంగా ఉంది.

---

10. At-most-once vs at-least-once – ఒక ఎంపిక, రెండు ఖర్చులు

§9 lo మనం retry చేర్చాం. దానితో ఒక కొత్త సమస్య వచ్చింది, మరియు దాన్ని ఒప్పుకోవాలి.

Handler **పని పూర్తిచేసి, ఆపై ack పంపేలోపు crash అయితే?** – Message in-flight lo ఉంది, timeout తర్వాత అది తిరిగి queue lo వస్తుంది, మరియు **మళ్ళీ ప్రాసెస్ అవుతుంది**. అంటే payment **రెండుసార్లు** జరుగుతుంది.

	At-most-once	At-least-once
ఎలా	ముందే ack, ఆపై ప్రాసెస్	ప్రాసెస్, ఆపై ack (మనం చేసింది)
Message పోతుందా?	**అవును** (crash అయితే)	లేదు
రెండుసార్లు వస్తుందా?	లేదు	**అవును** (ack కి ముందు crash)
ఎక్కడ సరైనది	Analytics, logs, metrics	Payment, order, inventory

"EXACTLY-ONCE" ఎందుకు లేదు

అందరూ కోరుకునేది "exactly once" – పోకూడదు, రెండుసార్లు రాకూడదు. Distributed system lo **అది సాధ్యం కాదు**, ఎందుకంటే "పని పూర్తయింది" మరియు "అది broker కి తెలిసింది" – ఈ రెండిటి మధ్య ఎప్పుడూ ఒక ఖాళీ ఉంటుంది, మరియు ఆ ఖాళీలో crash జరగొచ్చు.

ఆచరణలో చేసేది: at-least-once ఎంచుకుని, handler ని **idempotent** గా చేయడం – అంటే "రెండుసార్లు నడిపినా ఫలితం ఒకటే". Payment కి అది `paymentId` ని ఒక unique key గా వాడటం: రెండో సారి అదే id వస్తే, "ఇది ఇప్పటికే అయింది" అని వదిలేయడం.

Interview lo ఈ వాక్యం విలువైనది: *"Exactly-once delivery isn't achievable, so I'd do at-least-once and make the consumer idempotent. That moves the problem to a place where it's actually solvable."*

Part 5 – పూర్తి system

11. Step – Topics మరియు filters

ఇప్పటిదాకా **ప్రతి subscriber కి ప్రతి message** వెళ్తోంది. నిజ systems lo అది సరిపోదు:

- `order.placed`, `order.shipped`, `user.signup` – వేర్వేరు రకాల సంఘటనలు. Email service కి మొదటి రెండూ కావాలి, మూడోది కాదు.
- మరియు **ఒకే topic లోపల కూడా** filtering కావాలి – fraud check కి ₹10,000 దాటిన orders మాత్రమే.

Topic అనేది మొదటి, ముతక వడపోత. **Filter** అనేది రెండో, సూక్ష్మ వడపోత:

```
class Subscription {
  constructor(name, handler, { filter = () => true, maxAttempts = 3 } = {}) {
    Object.assign(this, { name, handler, filter, maxAttempts });
    this.dlq = [];
  }
  offer(msg) { if (this.filter(msg)) { this.#queue.push(msg); return true; } return
false; }
}
```

`offer` `false` **తిరిగి ఇస్తుంది** – filter తిరస్కరిస్తే. అది ముఖ్యం, ఎందుకంటే publisher కి "నలుగురు subscribers ఉన్నారు, కానీ ముగ్గురికే వెళ్ళింది" అని తెలుస్తుంది.

Filter ఎక్కడ నడవాలి – broker lo నా, subscriber lo నా? ఇది ఒక నిజమైన trade-off. Broker lo (మనం చేసింది) అంటే అనవసరమైన messages queue lo చేరవు – memory ఆదా. కానీ broker ఇప్పుడు message యొక్క లోపలి ఆకారం తెలుసుకోవాలి, అంటే అది కొంచెం తక్కువ సాధారణం.

Kafka **filter చేయదు** – consumer అన్నీ చదివి తనే వదిలేస్తాడు. SNS, RabbitMQ **చేస్తాయి**. రెండూ సరైనవే; **ఏది ఎందుకు ఎంచుకున్నారో చెప్పగలగడమే** ముఖ్యం.

12. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

```
class Message {
  constructor(id, topic, payload, key = null) {
    Object.assign(this, { id, topic, payload, key, attempts: 0 });
  }
}

class Subscription {
  #queue = []; #inFlight = new Map();
  constructor(name, handler, { filter = () => true, maxAttempts = 3 } = {}) {
```

```
    Object.assign(this, { name, handler, filter, maxAttempts });
    this.dlq = [];
  }
  offer(msg) { if (this.filter(msg)) { this.#queue.push(msg); return true; } return
false; }
  get pending()  { return this.#queue.length; }
  get inFlight() { return this.#inFlight.size; }

  poll() {
    const m = this.#queue.shift();
    if (m) this.#inFlight.set(m.id, m);        // తీసుకున్నాం, కానీ ఇంకా వదల్లేదు
    return m ?? null;
  }
  ack(id) { this.#inFlight.delete(id); }
  nack(id) {
    const m = this.#inFlight.get(id);
    if (!m) return null;
    this.#inFlight.delete(id);
    m.attempts++;
    if (m.attempts >= this.maxAttempts) { this.dlq.push(m); return 'DLQ'; }
    this.#queue.unshift(m);
    return 'RETRY';
  }
}

class Broker {
  #subs = new Map(); #seq = 0; #published = 0;

  subscribe(topic, name, handler, opts) {
    const s = new Subscription(name, handler, opts);
    if (!this.#subs.has(topic)) this.#subs.set(topic, []);
    this.#subs.get(topic).push(s);
    return s;
  }

  publish(topic, payload, key = null) {
    const targets = this.#subs.get(topic) ?? [];
    const msg = new Message(`M${++this.#seq}`, topic, payload, key);
    let queued = 0;
    for (const s of targets) if (s.offer({ ...msg, attempts: 0 })) queued++;
    this.#published++;
    return { messageId: msg.id, topic, queued, subscribers: targets.length };
  }

  deliver(topic, name) {
    const s = (this.#subs.get(topic) ?? []).find((x) => x.name === name);
    const stats = { ok: 0, retried: 0, dead: 0 };
    let m;
    while ((m = s.poll())) {
      try { s.handler(m); s.ack(m.id); stats.ok++; }
      catch {
        const r = s.nack(m.id);
        r === 'DLQ' ? stats.dead++ : stats.retried++;
        if (r === 'RETRY') break;              // తర్వాతి round lo మళ్ళీ ప్రయత్నిస్తాం
      }
    }
    return stats;
  }
```

```
  stats(topic) {
    return (this.#subs.get(topic) ?? []).map((s) =>
      ({ name: s.name, pending: s.pending, inFlight: s.inFlight, dlq: s.dlq.length
}));
  }
}
```

ప్రతి message ఒక copy ఎందుకు

`publish` lo `s.offer({ ...msg, attempts: 0 })` గమనించండి – ప్రతి subscriber కీ **ఒక సొంత copy**. ఎందుకు?

ఎందుకంటే `attempts` **subscriber వారీగా** ఉండాలి. Email subscriber ఒక message మీద రెండుసార్లు విఫలమైతే, అది analytics subscriber యొక్క లెక్కని తాకకూడదు. ఒకే object పంచుకుంటే వాళ్ళ retry counters కలిసిపోతాయి.

నాలుగు subscribers, ఒక filter, ఒక వైఫల్యం

```
broker.subscribe('order.placed', 'email', (m) => log.push(`email →
${m.payload.orderId}`));
broker.subscribe('order.placed', 'analytics', (m) => log.push(`analytics →
${m.payload.orderId}`));
broker.subscribe('order.placed', 'fraud-check',
                 (m) => log.push(`fraud → ${m.payload.orderId}`),
                 { filter: (m) => m.payload.amount > 10000 });      // ← ఖరీదైనవి
మాత్రమే
broker.subscribe('order.placed', 'payment', (m) => {
  if (m.payload.orderId === 'ORD-7') throw new Error('DB timeout');
  log.push(`payment → ${m.payload.orderId}`);
}, { maxAttempts: 3 });
```

```
publish ORD-6 (₹500): { messageId: 'M1', topic: 'order.placed', queued: 3,
subscribers: 4 }
publish ORD-7 (₹50000): { messageId: 'M2', topic: 'order.placed', queued: 4,
subscribers: 4 }

fraud-check ఎన్ని అందుకుంది? 1 ← filter పనిచేసింది (₹500 దాటలేదు)

deliver:
 email        { ok: 2, retried: 0, dead: 0 }
 analytics    { ok: 2, retried: 0, dead: 0 }
 fraud-check  { ok: 1, retried: 0, dead: 0 }
```

ఈ output ని పంక్తి పంక్తిగా చదువుదాం

`queued: 3, subscribers: 4` – ORD-6 (₹500) నలుగురిలో **ముగ్గురికే** వెళ్ళింది. Fraud check యొక్క filter దాన్ని తిరస్కరించింది. ORD-7 (₹50,000) కి `queued: 4` – నలుగురికీ వెళ్ళింది.

Email, analytics కి `ok: 2` – రెండు messages, రెండూ విజయవంతం. **Fraud-check కి** `ok: 1` – ఒక్కటే అందుకుంది.

ఇక్కడ ముఖ్యమైనది: payment subscriber ORD-7 మీద విఫలమవుతోంది, కానీ **email, analytics, fraud-check ముగ్గురూ ఆ message ని విజయవంతంగా ప్రాసెస్ చేశారు**. §4 యొక్క మొదటి విరుపు పూర్తిగా పరిష్కారమైంది – ఇప్పుడు ఒకరి వైఫల్యం ఇంకొకరిని తాకదు, మరియు వాళ్ళు వేరే వేగాలతో కూడా నడవగలరు.

```
విజయవంతంగా ప్రాసెస్ అయినవి:
  email → ORD-6
  email → ORD-7
  analytics → ORD-6
  analytics → ORD-7
  fraud → ORD-7
  payment → ORD-6
```

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	`subscribers`, `publish`	మౌలిక కలయిక విడదీత
§4 (విరుపు)	`try/catch` + `failures`	ఒకరి వైఫల్యం అందరినీ ఆపింది
§6 (విరుపు)	Per-subscriber queue	Publisher 415 ms ఆగాడు
§8 (విరుపు)	`poll` / `ack` / `nack`	Crash ఒక payment ని మాయం చేసింది
§9	`attempts`, `maxAttempts`, `dlq`	అపరిమిత retry queue ని ఆపేస్తుంది
§11	`topic`, `filter`	అందరికీ అన్నీ అవసరం లేదు

Part 6 — Interview lo

13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

ఈ problem lo code సులభం – విలువ అంతా "ఎందుకు" lo ఉంది

6 నిమి	**Clarify** – పోవచ్చా? రెండుసార్లు రావచ్చా? క్రమం?
4 నిమి	Observer vs broker తేడా · Basic bus
11 నిమి – Queue + async	isolation, మరియు publisher ఎందుకు ఆగకూడదో
14 నిమి – ack/retry/DLQ	**ఇదే ఈ interview యొక్క కేంద్రం**
10 నిమి	Delivery guarantees, ordering, scale

ఏమి తప్పక చెప్పాలి

1. **Observer vs broker** – మొదటి నిమిషంలోనే ఈ తేడాని చెప్పండి.
2. **Publisher subscriber వేగానికి బందీ కాకూడదు** (§6) – ఒక సంఖ్యతో ("415 ms").
3. **Ack తర్వాతే తీసేయడం** (§9) – **ఇదే కేంద్రం**. ఇది చెప్పకపోతే మిగతాదంతా వృథా.
4. **DLQ ఎందుకు** (§9) – అపరిమిత retry ఒక poison message ని శాశ్వతం చేస్తుంది.
5. **Exactly-once సాధ్యం కాదు; at-least-once + idempotent** (§10).

ఏమి వదిలేయాలి

- **Filter రామొద్దు** – ఒక్క వాక్యంలో చెప్పండి.
- `stats()` **వదిలేయండి.**
- **Message class వదిలేయండి** – ఒక plain object చాలు, మరియు `attempts` ఎక్కడ ఉండాలో చెప్తే చాలు.

14. నోటితో చెప్పాల్సిన English script

"Two questions that shape everything here. Can a message be lost, and can it be delivered twice? You can't say no to both – exactly-once isn't achievable in a distributed system – so I need to know which side to err on. And does ordering matter?

I'll assume messages must not be lost, duplicates are tolerable if consumers are idempotent, and ordering matters per entity but not globally.

The simplest version is a list of subscribers and a loop. That already fixes coupling – the order service stops knowing about email and inventory. But it breaks in three ways, and each one is worth fixing separately.

First, if one subscriber throws, the loop stops and everyone after it never gets the message – and the exception surfaces in the publisher, so a broken SMTP server makes `placeOrder` look like it failed when the order actually saved. So each handler gets its own try-catch and failures are collected and returned rather than thrown.

Second, even isolated, the publisher still pays for everyone's time. I measured it – one fraud check taking 400 ms made publish take 415. That means every new subscriber slows down checkout, which kills the whole point of pub-sub. So publish shouldn't call handlers at all; it should enqueue, and workers drain the queues separately. That took publish from 415 ms to zero.

Third – and this is the one I care about most – once it's a queue, you have to be careful about when you remove a message. If you pop it and then the handler crashes, that message is gone with no trace. I saw a payment vanish that way in my own test. So `poll` moves it to an in-flight set rather than dropping it, and it's only really removed on `ack`. On failure, `nack` puts it back and increments an attempt count.

That needs a bound, though. Unlimited retries mean one poison message spins forever and blocks everything behind it. So after N attempts it goes to a dead letter queue – that's the system saying 'this isn't transient, a human should look'.

On guarantees: acking after processing gives at-least-once, so a crash between finishing and acking means a redelivery. I'd accept that and make consumers idempotent – for payments, keying on a payment id so a repeat is a no-op. That moves the problem somewhere it's actually solvable.

One thing I'd flag: each subscriber needs its own copy of the message, because the attempt count is per-subscriber. Sharing one object would merge their retry state."

15. Follow-ups – ordering, fan-out, Kafka

Follow-up	జవాబు	మారే classes
"Subscriber నెమ్మదిగా ఉంటే queue పెరుగుతుంది"	Queue కి ఒక హద్దు + backpressure, లేదా పాతవి వదిలేయడం	Subscription
"ఒకే subscriber కి పలు workers"	Queue ఒకటే, workers పలుగురు – poll ఇప్పటికే సరైనది	**0**
"Priority messages"	Queue ని ఒక priority queue గా	Subscription
"Retry వెంటనే కాకుండా ఆలస్యంగా"	`nack` lo `availableAt = now + backoff`	Subscription
"Subscriber కొంతసేపు ఆఫ్‌లైన్ అయితే"	Queue అలాగే ఉంటుంది – అదే durable subscription యొక్క అర్థం	**0**
"క్రమం కావాలి"	కింద చూడండి	–
"లక్షల messages, పలు servers"	కింద చూడండి	–

క్రమం (ordering)

> *"Global ordering ఖరీదైనది మరియు దాదాపు ఎప్పుడూ అవసరం లేదు. నిజంగా కావలసినది **per-entity ordering** – ఒకే order యొక్క 'created' మరియు 'shipped' సరైన క్రమంలో రావాలి; వేర్వేరు orders మధ్య క్రమం అనవసరం.*
>
> *అందుకే నా `Message` lo ఒక `key` ఉంది. ఒకే key ఉన్న messages ఒకే queue (లేదా partition) కి వెళ్తాయి, మరియు ఆ queue ని ఒక్క worker మాత్రమే ప్రాసెస్ చేస్తాడు. వేరే keys సమాంతరంగా నడుస్తాయి. **ఇదే Kafka partitioning.***
>
> *మరియు గమనించండి – retry lo నేను `unshift` వాడాను, `push` కాదు. విఫలమైన message **ముందుకే** తిరిగి వెళ్తుంది, లేకపోతే దాని వెనక ఉన్నవి ముందుకు దూరి క్రమం విరిగిపోతుంది."*

లక్షల messages, పలు servers

> *"In-memory queue ఒకే process కే. Production lo నేను Kafka లేదా SQS వాడతాను, మరియు నా design అదే ఆకారంలో ఉంది – topic, partition key, offset/ack, DLQ. ఇవన్నీ ఇక్కడ ఉన్నవే.*
>
> ***ఒక ముఖ్యమైన తేడా:** Kafka lo broker messages ని తీసేయదు – consumer ఒక **offset** ఉంచుకుంటాడు. అంటే ఒక కొత్త consumer వచ్చి **మొదటి నుంచి** అన్నీ చదవగలడు. నా in-memory version lo అది సాధ్యం కాదు, ఎందుకంటే ack తర్వాత message పోతుంది. ఆ తేడా ముఖ్యం – Kafka ఒక **log**, SQS ఒక **queue**, మరియు అవి వేర్వేరు పనులకి."*

16. ఏమి నేర్చుకున్నాం

ఆలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
వైఫల్యాన్ని వేరుచేయడం	Per-subscriber try/catch (§5)	Bulkhead, circuit breaker, thread pools
ప్రకటించడం ≠ ప్రాసెస్ చేయడం	Queue (§7)	Async APIs, background jobs, outbox
పని పూర్తయ్యాకే వదలడం	poll → ack (§9)	DB transactions, file writes, 2PC
Retry కి ఒక హద్దు	DLQ (§9)	ఏ retry lo అయినా
Exactly-once సాధ్యం కాదు	At-least-once + idempotent (§10)	Payments, webhooks, ETL
క్రమం per-key, global కాదు	`key` → partition (§15)	Kafka, sharding, sequencing

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot** · **02 LRU & LFU Cache** · **03 Rate Limiter** · **04 BookMyShow** · **05 Splitwise** · **06 Elevator** · **07 Pub-Sub**.

ఈ doc ప్రత్యేకత: మిగతావాటిలో విరుపులు తప్పు జవాబు ఇచ్చాయి. ఇక్కడ మూడో విరుపు – మాయమైన payment – **ఏ error నీ ఇవ్వలేదు**. System "అంతా బాగుంది" అని చెప్పింది. **నిశ్శబ్దంగా విఫలమయ్యే systems** అత్యంత ప్రమాదకరమైనవి, మరియు ఆ ఆలోచనే మిగతా అన్నిటికీ వర్తిస్తుంది.

వేగవంతమైన revision కోసం – `LLD_Design_Problems_Telugu.pdf` lo Problem 10. Observer pattern కోసం – `LLD_Telugu.pdf` §27.

Pub-Sub – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output, ప్రతి timing నిజంగా `node` lo run చేసి తీసినవే ✅